

Fullstack Lab Report – Recipe & Nutrition Application

Author: Morgan Lindbom

<https://github.com/morganlindbom/mogges-bars>

1. System Overview

Purpose;

The application is a fullstack web system for creating and analyzing nutrition-based recipes. Users can combine ingredients, assign quantities, and automatically calculate macronutrients and total calories.

Target Users:

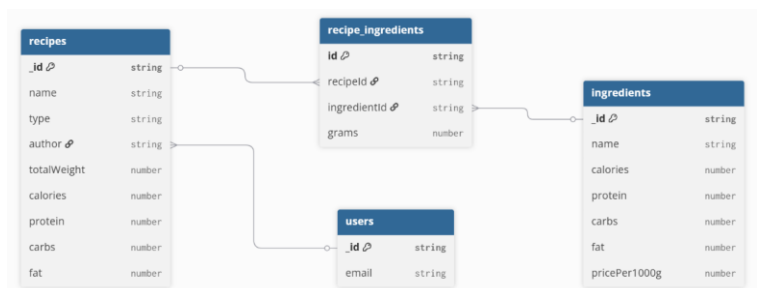
The system is intended for users who want to understand and optimize their diet, such as fitness enthusiasts or individuals tracking their nutritional intake.

Value:

The application provides a structured way to compare recipes by normalizing nutritional values per 100 grams and visualizing the results, helping users make informed dietary decisions.

2. Database Design

The database consists of users, recipes, and ingredients. Recipes reference users and contain ingredient entries. A many-to-many relationship is modeled using a linking structure.



3. API Endpoints

Endpoint 1

Route: `/api/recipes`

Method: POST

Auth: Required

Purpose: Create recipe

Request:

`{name, type, ingredients[]}`

Validation:

`name, type, grams > 0`

Response:

`{_id, name, macros, ingredients}`

Endpoint 2

Route: `/api/recipes/:id`

Method: GET

Auth: Not required

Purpose: Get recipe by ID

Request:

None

Response:

`{_id, name, macros, ingredients}`

Errors:

404 if not found

4. Reflection

Describe one challenge you faced.

One challenge I faced was implementing the update functionality for recipes. When attempting to update a recipe, the application returned a 404 error, meaning the endpoint could not be found.

Explain how you solved it.

The issue was caused by a mismatch between the frontend request and the backend implementation. The PUT route for updating recipes was either missing or not properly connected to the controller and service. I resolved this by adding the correct route, implementing the update logic in the controller and service, and ensuring that the request structure matched the backend validation.

Explain what you learned.

Through this process, I learned the importance of keeping frontend and backend aligned, especially when working with APIs. I also gained a better understanding of debugging REST endpoints and the value of separating responsibilities between routes, controllers, and services.

5. Feature Iteration

Describe one feature that changed during development.

One feature that evolved during development was the recipe creation and management system. Initially, the feature only supported basic creation of recipes with manually entered data.

Reference at least two commits.

In commit `ba15f03`, the initial bar-ingredient system was implemented, allowing users to create recipes by selecting ingredients and assigning quantities. This introduced the core functionality of the application.

In commit `08c47d5`, the data model was standardized, improving consistency between frontend and backend and making the system more stable.

In commit `bf13123`, authentication validation and request handling were improved, ensuring that only authorized users could create and manage recipes.

Explain how the feature improved.

These changes transformed the feature from a basic prototype into a more robust and secure system, with improved data consistency, better user handling, and a clearer separation of responsibilities across the application.